

In the above section we have described a class, i. e, Jack and the associated method, sleep(). The code for the sleep() method defines all the details of the method (how Jack sleeps, etc). When encapsulated, this information cannot be revealed to others. We can protect this information from other classes by using the system of encapsulation. By using this system we can easily edit our web pages where the clients' status will not be changed as some certain data will always be hidden and protected from the clients.

Polymorphism

Polymorphism is a unique feature of OOPs. By using this feature we can use a particular interface for a generalised class of actions. The exact nature of the situation can determine the specific action. In a nutshell we can say that polymorphism is a system that includes a single interface and multiple methods. We can design a general interface to a group that includes multiple activities. Many complexities can be lessened as here we can apply a single interface for multiple actions. The 'method' can specify the actions that are to be applied via the common interface.

Inheritance

Inheritance delivers us a strong and powerful system to structure our desired software. The classes of the programming language usually inherit their state and behaviour from super-classes. Inheritance is nothing but the derivation of one class from another class.

Let us explain this inheritance a bit:

Nowadays, a single object or a commodity is frequently updated in the course of time. At the same time a lot of variation of a single object is also available due to enormous market competition. Say you want to club some objects that are similar to some extent or that are nothing but the variation of a single object. So here by applying the theory of inheritance you can define the parent object as the super-class and the variations of the parent object as the sub-class.